

GPUMODE Lecture Study Notes: Lecture 80

How FlashAttention 4 Works

Core Problem the Lecture Addresses

This lecture explains how **FlashAttention 4** adapts exact attention to NVIDIA Blackwell-class GPUs, especially B200-class SM100 or SM100 hardware. The core problem is not merely computing attention correctly. The real problem is computing attention while keeping modern tensor cores busy, avoiding unnecessary high-bandwidth memory traffic, preserving softmax numerical stability, and managing an increasingly asynchronous GPU programming model.

For one attention head, the mathematical operation is

$$O = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right) V,$$

where

$$Q \in \mathbb{R}^{N_q \times d}, \quad K \in \mathbb{R}^{N_k \times d}, \quad V \in \mathbb{R}^{N_k \times d_v}, \quad O \in \mathbb{R}^{N_q \times d_v}.$$

Here N_q is the number of query positions, N_k is the number of key-value positions, d is the head dimension, and d_v is the value dimension.

The hardware-naïve implementation materializes

$$S = \frac{QK^T}{\sqrt{d}},$$

then computes

$$P = \text{softmax}(S),$$

and finally computes

$$O = PV.$$

This is simple mathematically, but bad for GPU performance because the intermediate matrix S has shape $N_q \times N_k$. For long sequences, this matrix is enormous and must be written to and read from high-bandwidth memory if the operations are implemented as separate kernels.

The FlashAttention family avoids this by streaming over tiles of K and V , keeping intermediate state in faster on-chip storage, and computing the softmax online. FlashAttention 4 keeps this general philosophy but changes the implementation to match Blackwell:

- It uses a much more specialized warp pipeline.
- It uses Blackwell tensor memory.
- It relies on asynchronous cooperation among specialized warps.
- It improves online softmax rescaling.
- It sometimes replaces hardware exponentiation on special function units with software exponentiation on CUDA cores.
- It is written using newer tile-oriented DSL machinery such as CuTe DSL and CUTLASS Python rather than being a traditional CUDA C++ kernel.

The speaker frames FlashAttention 4 as the first **petaflop-scale** kernel in the FlashAttention series. It is roughly about twice as fast as FlashAttention 3 on H100 in the discussed comparison, and around fifteen times faster than the original FlashAttention implementation on A100. The performance is described as roughly around the familiar high-performance attention range of 30% to 50% model FLOP utilization, with FlashAttention 4 reaching petaflop-scale throughput on B200.

Required Background

Scaled Dot-Product Attention

Attention compares each query vector q_i against all key vectors k_j . The score between query i and key j is

$$s_{ij} = \frac{q_i \cdot k_j}{\sqrt{d}}.$$

The softmax converts scores into probabilities:

$$p_{ij} = \frac{\exp(s_{ij})}{\sum_{\ell=1}^{N_k} \exp(s_{i\ell})}.$$

The output vector for query i is

$$o_i = \sum_{j=1}^{N_k} p_{ij} v_j.$$

The operation is dominated by two matrix multiplications:

1. QK^T , which produces attention scores.

2. *PV*, which applies attention probabilities to values.

The key challenge is the softmax in the middle. Matrix multiplication is tensor-core-friendly. Softmax is not. It requires exponentials, reductions, and careful numerical stabilization.

Numerically Stable Softmax

The naive softmax

$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

is numerically unsafe because $\exp(z_i)$ can overflow when z_i is large.

The standard numerically stable form subtracts the maximum:

$$\text{softmax}(z_i) = \frac{\exp(z_i - m)}{\sum_j \exp(z_j - m)}, \quad m = \max_j z_j.$$

This works because subtracting the same scalar from every score does not change the softmax:

$$\frac{\exp(z_i - m)}{\sum_j \exp(z_j - m)} = \frac{\exp(z_i) \exp(-m)}{\sum_j \exp(z_j) \exp(-m)} = \frac{\exp(z_i)}{\sum_j \exp(z_j)}.$$

The problem is that computing m over a full row of QK^T seems to require seeing the entire row before computing the softmax. FlashAttention solves this by maintaining the row maximum online while streaming over tiles.

GPU Execution Model

A CUDA program is launched as a **grid** of thread blocks. Each thread block, also called a cooperative thread array or CTA, runs on one streaming multiprocessor.

Important execution concepts:

- **Thread**: The smallest CUDA programming unit.
- **Warp**: A group of 32 threads that execute together.
- **Thread block or CTA**: A group of threads guaranteed to reside on the same SM.
- **SM**: A streaming multiprocessor containing warp schedulers, registers, shared memory, tensor cores, CUDA cores, and special function units.
- **Grid**: A collection of CTAs launched by a kernel.

For high-performance attention, the important unit is not an individual thread but a tile-processing CTA. FlashAttention maps tiles of Q and O to CTAs. Each CTA streams through tiles of K and V .

Memory Hierarchy

A simplified GPU memory hierarchy is

Registers $\rightarrow$ Tensor Memory / Accumulators $\rightarrow$ Shared Memory $\rightarrow$ L1/L2 Cache $\rightarrow$ HBM.

Approximate intuition:

- Registers are fastest and private to threads.
- Shared memory is on-chip and visible to a CTA.
- Tensor memory on Blackwell is a new accumulator-like storage level associated with tensor core outputs.
- L2 cache is shared across SMs.
- HBM is large but much slower and bandwidth-limited relative to on-chip storage.

The central optimization principle of FlashAttention is:

Move less data through HBM, and reuse data while it is on-chip.

Arithmetic Intensity and Roofline Intuition

Arithmetic intensity is

$$\text{Arithmetic Intensity} = \frac{\text{FLOPs}}{\text{Bytes Moved}}.$$

A kernel is memory-bound when

$$\text{Achievable FLOPs} \approx \text{Memory Bandwidth} \times \text{Arithmetic Intensity}.$$

A kernel is compute-bound when it approaches the hardware's peak compute throughput:

$$\text{Achievable FLOPs} \approx \text{Peak Compute Throughput}.$$

FlashAttention increases arithmetic intensity by avoiding the materialized attention matrix. It computes tiles of scores and immediately consumes them to update the output, reducing HBM traffic.

Main Concepts

What FlashAttention 4 Is

FlashAttention 4 is a forward-pass attention kernel designed for Blackwell B200-class GPUs. In the lecture, the speaker emphasizes that it is still early and

still evolving. The discussed implementation is specifically a forward kernel, with backward support promised later. It currently focuses on BF16 inputs and outputs, with lower-precision versions expected later. It also supports paged key-value tensors, which are important for KV caches in inference, and supports attention sinks, which are used for long-context handling.

Conceptually, FlashAttention 4 is best interpreted as a highly optimized prefill or training-oriented forward attention kernel rather than a final decode-only kernel. It operates at BF16 precision, while much inference deployment often uses lower precision such as FP8, INT8, or other compressed formats.

FlashAttention 1: Tiling and Online Softmax

FlashAttention 1 applied tiling to attention. Instead of writing

$$S = QK^T$$

to HBM, it computes tiles of S , applies softmax logic online, and updates the output tile.

For one row, suppose we process score blocks $S^{(1)}, S^{(2)}, \dots, S^{(T)}$. For each row, maintain:

$$m_t = \max(m_{t-1}, \max_j S_j^{(t)}),$$

$$\ell_t = \sum_{j \leq \text{seen}} \exp(S_j - m_t),$$

and

$$o_t = \sum_{j \leq \text{seen}} \exp(S_j - m_t) v_j.$$

When the running maximum changes from m_{t-1} to m_t , the old accumulated terms must be rescaled:

$$\exp(S_j - m_t) = \exp(S_j - m_{t-1}) \exp(m_{t-1} - m_t).$$

Thus the denominator update is

$$\ell_t = \ell_{t-1} \exp(m_{t-1} - m_t) + \sum_{j \in t} \exp(S_j^{(t)} - m_t),$$

and the numerator update is

$$o_t = o_{t-1} \exp(m_{t-1} - m_t) + \sum_{j \in t} \exp(S_j^{(t)} - m_t) v_j.$$

The final output is

$$O = \frac{o_T}{\ell_T}.$$

The key insight is that exact softmax can be computed online without materializing the full score matrix.

FlashAttention 2: Better Work Partitioning

FlashAttention 2 changed the way work is partitioned. The speaker describes the dominant pattern in FlashAttention 2 onward as:

Load a query tile, scan through all key tiles, scan through all value tiles, and produce one output tile.

A CTA owns a tile of Q and the corresponding tile of O . It streams over all K and V tiles required for that query tile. This is natural because the output for a given query tile depends on that query tile and all relevant keys and values, but it does not depend on other query tiles.

If Q_b is a block of queries, then the CTA computes

$$O_b = \text{softmax}\left(\frac{Q_b K^T}{\sqrt{d}}\right) V.$$

The CTA loops over key-value blocks:

$$K = \begin{bmatrix} K_1 \\ K_2 \\ \vdots \\ K_T \end{bmatrix}, \quad V = \begin{bmatrix} V_1 \\ V_2 \\ \vdots \\ V_T \end{bmatrix}.$$

For each t , it computes partial scores

$$S_t = \frac{Q_b K_t^T}{\sqrt{d}},$$

updates the online softmax state, and accumulates output contributions from V_t .

FlashAttention 3: Asynchrony and Warp Specialization

FlashAttention 3 targeted Hopper GPUs and introduced more explicit asynchrony. Two important Hopper features were:

- **Tensor Memory Accelerator:** Asynchronous hardware for loading multidimensional tensor tiles.
- **Asynchronous matrix multiply-accumulate:** The program can issue tensor core operations and later wait for results.

FlashAttention 3 also used **warp specialization**. In a warp-specialized kernel, different warps inside a CTA perform different roles. A simplified FlashAttention 3 decomposition is:

- **Producer warps:** Load Q , K , and V tiles into shared memory.
- **Consumer warps:** Perform tensor-core matrix multiplications, softmax operations, and output accumulation.

This is a producer-consumer pipeline:

HBM $\rightarrow$ Shared Memory $\rightarrow$ Tensor Cores and CUDA Cores $\rightarrow$ Output.

Dynamic register allocation on Hopper made warp specialization more useful. Producer warps that mostly issue copies need fewer registers, while consumer warps need more registers for computation and accumulation.

FlashAttention 4: More Specialized Pipeline

FlashAttention 4 extends the FlashAttention 3 idea but makes the pipeline more elaborate. Instead of a simple producer-consumer split, the lecture describes approximately five classes of specialized warp roles:

- **Load warp:** Loads input tiles into shared memory.
- **MMA warp:** Performs matrix multiply-accumulate operations for QK^T and PV .
- **Softmax warps:** Apply softmax transformations and determine when rescaling is needed.
- **Correction warps:** Rescale accumulated output state when the softmax running maximum changes enough.
- **Store warp:** Writes output tiles back to global memory.

The high-level dataflow is

$(Q, K, V)_{\text{HBM}} \rightarrow \text{Load Warp} \rightarrow \text{Shared Memory} \rightarrow \text{MMA Warp} \rightarrow \text{Tensor Memory} \rightarrow \text{Softmax and Correction}$

A simplified pipeline view is:

Load $\rightarrow QK^T$ MMA $\rightarrow$ Softmax $\rightarrow$ Correction $\rightarrow PV$ MMA $\rightarrow$ Store.

The exact implementation is more complicated because a CTA may work on two query-output tiles at once, using circular buffers and barriers to coordinate movement among stages.

Visual Concept

Draw one CTA as a horizontal pipeline. On the left, show HBM feeding Q , K , and V tiles into shared memory through load warps. In the middle, show an MMA warp producing score tiles into tensor memory. Next, show softmax and correction warps operating on the score state. On the right, show a store warp writing O back to HBM.

Hardware-Level Explanation

Blackwell Tensor Memory

A central Blackwell feature discussed in the lecture is **tensor memory**. Despite the similar name, tensor memory is different from the Tensor Memory Accelerator. Tensor memory is described as an accumulator-like storage location associated with tensor core outputs.

For attention, tensor memory is useful because QK^T produces intermediate score tiles, and those score tiles are consumed by softmax and then by the second matrix multiplication with V . Instead of treating all intermediate state as ordinary registers or shared memory, Blackwell exposes tensor memory as another storage level for tensor-core-centric kernels.

A useful mental hierarchy is:

HBM $\rightarrow$ Shared Memory $\rightarrow$ Tensor Memory $\rightarrow$ Tensor Core Computation.

Tensor memory is not a general-purpose replacement for shared memory. It is closer to a tensor-core-facing accumulator space. In FlashAttention 4, it helps manage the intermediate tensors produced by matrix multiply operations.

Why Warp Groups Matter Less on Blackwell

On Hopper, fast tensor core operations often used **warp groups**, where four consecutive warps cooperated on matrix multiply operations. FlashAttention 3 therefore had to reason carefully about warp-group MMA execution.

On Blackwell, the lecture states that FlashAttention 4 drops the Hopper-style warp-group emphasis because Blackwell fifth-generation tensor core instructions can be driven differently. A single warp can issue relevant tensor core work more directly, simplifying part of the MMA programming model.

This does not mean the kernel is simple. Instead, the complexity moves toward:

- More pipeline stages.
- More specialized warp roles.
- More explicit barriers.
- More circular-buffer management.
- More fine-grained scheduling inside the CTA.

Special Function Units and Exponentiation

Softmax requires exponentiation:

$$\exp(x).$$

On NVIDIA GPUs, exponentiation is commonly implemented using a base-2 exponential intrinsic:

$$\exp(x) = 2^{x \log_2(e)}.$$

The intrinsic typically maps to special function units, or SFUs.

SFUs are much less abundant than CUDA cores and tensor cores. If a kernel performs many exponentials, it can stall waiting for SFU throughput. FlashAttention 4 therefore includes a software-emulated exponentiation path that can move some exponential work from SFUs to CUDA cores.

The core idea is:

$$2^x = 2^{\lfloor x \rfloor} 2^{x - \lfloor x \rfloor}.$$

The integer part $2^{\lfloor x \rfloor}$ is handled by exponent-field manipulation or an equivalent scale operation. The fractional part

$$r = x - \lfloor x \rfloor, \quad r \in [0, 1),$$

is approximated by a low-degree polynomial:

$$2^r \approx a_0 + a_1 r + a_2 r^2 + a_3 r^3.$$

A cubic polynomial can be evaluated with fused multiply-add operations:

$$p(r) = ((a_3 r + a_2) r + a_1) r + a_0.$$

This uses CUDA-core-friendly FMA instructions rather than SFU exponent instructions.

In the lecture, the speaker points out that the code uses inline PTX to evaluate pairs of values with packed instructions and a small number of FMA operations. This is described as software emulation of the hardware exponent path, with bit-identical results at the BF16 precision level for the relevant range.

Why Software Exponentiation Is Conditional

FlashAttention 4 does not necessarily replace every exponentiation with the software path. The lecture describes conditions such as a frequency and a fragment limit. The interpretation is that software exponentiation is useful only when SFUs are the bottleneck. Near the end of a kernel, wave quantization effects may leave less work in flight, reducing SFU pressure. In that region, the ordinary SFU path may be adequate.

Thus the strategy is:

Use software exponentiation when SFU pressure is high, otherwise use hardware exponentiation.

This is a hardware-aware scheduling choice. It does not change the attention algorithm mathematically, but it changes which execution units are used.

Asynchrony and Barriers

FlashAttention 4 relies heavily on programmer-managed asynchrony. This means the programmer is responsible for coordinating when data is ready, when buffers are free, and when computation can safely proceed.

If one warp writes a shared-memory tile and another warp reads it, then the program must enforce:

write completes before read begins.

If a buffer is reused, it must also enforce:

read completes before overwrite begins.

This requires barriers or barrier-like synchronization primitives. Incorrect ordering can cause:

- Reading stale data.
- Overwriting data too early.
- Deadlock.
- Invalid pipeline state.

The speaker compares this style to writing an event loop rather than using a high-level `async/await` abstraction. The programmer directly manages the pipeline.

Mathematical Reasoning

Naive Attention Memory Cost

Materializing $S = QK^T$ requires storing

$$N_q N_k$$

scores. If each score uses b bytes, the storage cost is

$$\text{Bytes}(S) = b N_q N_k.$$

For long-context attention with $N_q = N_k = N$,

$$\text{Bytes}(S) = b N^2.$$

This quadratic intermediate is the main memory problem.

FlashAttention avoids writing S to HBM. Instead, it computes score tiles:

$$S_{bt} = \frac{Q_b K_t^T}{\sqrt{d}},$$

where Q_b is a query block and K_t is a key block.

Online Softmax State

For a row of scores streamed in blocks, define the state after processing block t :

$$\begin{aligned}m_t &= \max_{j \leq t} s_j, \\ \ell_t &= \sum_{j \leq t} \exp(s_j - m_t), \\ u_t &= \sum_{j \leq t} \exp(s_j - m_t) v_j.\end{aligned}$$

The output after all blocks is

$$o = \frac{u_T}{\ell_T}.$$

Suppose a new block has local maximum

$$\tilde{m}_t = \max_{j \in B_t} s_j.$$

Then the new maximum is

$$m_t = \max(m_{t-1}, \tilde{m}_t).$$

The old denominator must be rescaled:

$$\ell'_{t-1} = \ell_{t-1} \exp(m_{t-1} - m_t).$$

The new block contribution is

$$\tilde{\ell}_t = \sum_{j \in B_t} \exp(s_j - m_t).$$

Thus

$$\ell_t = \ell'_{t-1} + \tilde{\ell}_t.$$

Similarly,

$$u_t = u_{t-1} \exp(m_{t-1} - m_t) + \sum_{j \in B_t} \exp(s_j - m_t) v_j.$$

This is the mathematical backbone of FlashAttention.

FlashAttention 4 Deferred Rescaling

The lecture highlights a major FlashAttention 4 softmax improvement: it does not necessarily rescale every time the running maximum increases slightly. Rescaling is needed for numerical safety, not because the mathematical softmax inherently requires rescaling at every tiny increase.

Let

$$\Delta_t = m_t - m_{t-1}.$$

Classic online softmax rescales whenever

$$\Delta_t > 0.$$

FlashAttention 4 introduces a threshold τ and rescales only when

$$\Delta_t > \tau.$$

Equivalently, if the new maximum increases only slightly,

$$0 < \Delta_t \leq \tau,$$

the kernel can defer correction because the exponentials remain numerically safe within the chosen BF16 error tolerance.

The rescale factor is

$$\alpha_t = \exp(m_{t-1} - m_t) = \exp(-\Delta_t).$$

If Δ_t is small, then

$$\alpha_t \approx 1 - \Delta_t.$$

For sufficiently small Δ_t , the correction may not matter enough at BF16 precision to justify a synchronization-heavy rescale.

The practical benefit is fewer correction events:

$$\text{Number of corrections}_{\text{FA4}} \ll \text{Number of corrections}_{\text{classic online softmax}}.$$

The lecture mentions that the reduction can be up to roughly $10\times$ in the number of rescale operations when the threshold is set appropriately.

Why Rescaling Is Expensive

A rescale is not just one multiply. In a warp-specialized pipeline, the softmax warp detects the need for correction, but correction warps perform the rescaling. Therefore a correction requires:

1. Detecting that the maximum changed enough.
2. Communicating this event across warp roles.
3. Synchronizing relevant stages.
4. Rescaling previous accumulated output state.
5. Ensuring dependent work does not proceed with stale scaling.

Thus the cost is closer to

$$\text{Cost}_{\text{rescale}} = \text{FMA cost} + \text{coordination cost} + \text{synchronization cost} + \text{pipeline delay}.$$

Reducing the number of rescale events can have significant impact even if the arithmetic itself is cheap.

Software Exponential Approximation

The software exponent path approximates

$$2^x.$$

Decompose

$$x = n + r, \quad n = \lfloor x \rfloor, \quad r \in [0, 1).$$

Then

$$2^x = 2^n 2^r.$$

Approximate 2^r by a cubic:

$$2^r \approx c_0 + c_1 r + c_2 r^2 + c_3 r^3.$$

Using Horner form:

$$2^r \approx (((c_3 r + c_2) r + c_1) r + c_0).$$

This maps well to fused multiply-add:

$$y_1 = c_3 r + c_2,$$

$$y_2 = y_1 r + c_1,$$

$$y_3 = y_2 r + c_0.$$

The final result is

$$2^x \approx 2^n y_3.$$

Since softmax usually needs

$$\exp(s - m),$$

the implementation commonly computes

$$\exp(s - m) = 2^{(s-m) \log_2(e)}.$$

Performance Model for Attention

The two main matrix products have approximate FLOP counts:

$$\text{FLOPs}(QK^T) \approx 2N_q N_k d,$$

$$\text{FLOPs}(PV) \approx 2N_q N_k d_v.$$

If $d_v = d$, the attention matrix multiplication work is roughly

$$\text{FLOPs}_{\text{attention matmuls}} \approx 4N_q N_k d.$$

Softmax adds non-matmul work:

$$\text{Softmax work} \approx N_q N_k (\text{max comparisons} + \text{subtractions} + \text{exponentials} + \text{reductions} + \text{normalization}).$$

Tensor cores handle the matrix products, while CUDA cores and SFUs handle softmax. On Blackwell, as tensor core throughput increases, the non-tensor-core portion becomes more visible. This explains why FlashAttention 4 pays attention to softmax rescaling and exponentiation.

Code Walkthrough

Naive Attention in PyTorch

A straightforward implementation is:

```
import torch

def naive_attention(q, k, v):
    d = q.shape[-1]
    scores = q @ k.transpose(-2, -1)
    scores = scores / (d ** 0.5)
    probs = torch.softmax(scores, dim=-1)
    out = probs @ v
    return out
```

Line-by-line explanation:

- `scores = q @ k.transpose(-2, -1)` computes QK^T .
- `scores = scores / (d ** 0.5)` applies the transformer scaling factor.
- `torch.softmax(scores, dim=-1)` normalizes each row.
- `out = probs @ v` computes PV .

This is readable but inefficient for long sequences because `scores` and `probs` are full $N_q \times N_k$ tensors. They are written to and read from HBM, causing high memory traffic.

Representative Online Attention Pseudocode

A simplified online tiled attention algorithm is:

```
def online_attention_block(q_block, k_blocks, v_blocks, scale):
    m = -float("inf")
    l = 0.0
    acc = 0.0

    for k_block, v_block in zip(k_blocks, v_blocks):
        scores = (q_block @ k_block.T) * scale

        block_m = scores.max(axis=-1)
        new_m = maximum(m, block_m)

        old_scale = exp(m - new_m)
        new_scale = exp(scores - new_m[:, None])

        acc = acc * old_scale[:, None] + new_scale @ v_block
        l = l * old_scale + new_scale.sum(axis=-1)
```

```

    m = new_m

    out = acc / l[:, None]
    return out

```

This captures the essential online softmax state:

$$m \leftrightarrow \mathbf{m}, \quad \ell \leftrightarrow \mathbf{l}, \quad u \leftrightarrow \mathbf{acc}.$$

Hardware interpretation:

- `q_block @ k_block.T` maps to tensor core MMA.
- `scores.max` is a reduction over the key tile.
- `exp` usually maps to SFU instructions unless software exponentiation is used.
- `new_scale @ v_block` maps to the second tensor core MMA if represented in a tensor-core-friendly layout.
- The loop streams through K, V blocks without materializing the full score matrix.

Deferred Rescaling Pseudocode

FlashAttention 4's thresholded rescaling idea can be approximated as:

```

def should_rescale(old_m, new_m, threshold):
    delta = new_m - old_m
    return delta > threshold

```

A more representative algorithm sketch is:

```

def online_attention_deferred_rescale(q_block, k_blocks, v_blocks, scale, threshold):
    m = -float("inf")
    effective_m = -float("inf")
    l = 0.0
    acc = 0.0

    for k_block, v_block in zip(k_blocks, v_blocks):
        scores = (q_block @ k_block.T) * scale
        block_m = scores.max(axis=-1)
        candidate_m = maximum(m, block_m)

        if any(candidate_m - effective_m > threshold):
            correction = exp(effective_m - candidate_m)
            acc = acc * correction[:, None]
            l = l * correction

```

```

effective_m = candidate_m

weights = exp(scores - effective_m[:, None])
acc = acc + weights @ v_block
l = l + weights.sum(axis=-1)
m = candidate_m

out = acc / l[:, None]
return out

```

This code is not the real FlashAttention 4 implementation. It is a conceptual reconstruction. The actual kernel uses warp-level masks, barriers, specialized warp roles, tensor memory, shared memory, and inline PTX.

The key hardware point is that checking whether to rescale can be cheaper than performing the actual correction because correction requires coordination among different warp roles.

Software Exponentiation Pseudocode

A conceptual software approximation for 2^x is:

```

def exp2_approx(x):
    n = floor(x)
    r = x - n

    c0 = 1.0
    c1 = 0.69314718056
    c2 = 0.24022650695
    c3 = 0.05550410866

    frac = ((c3 * r + c2) * r + c1) * r + c0
    return ldexp(frac, int(n))

```

The constants above are illustrative and not claimed to be the exact constants used by FlashAttention 4. The important idea is that polynomial approximation can be implemented using CUDA-core FMA operations. The real kernel uses carefully chosen constants and inline assembly to achieve the desired precision behavior.

Hardware interpretation:

- `floor` and fractional extraction split scale from shape.
- The cubic polynomial uses FMA operations.
- The final scaling by 2^n is cheap relative to a transcendental function.
- This can reduce pressure on SFUs.

Warp Specialization Sketch

A simplified CUDA-style sketch of warp specialization is:

```
__global__ void fa4_like_kernel(...) {
    int tid = threadIdx.x;
    int warp_id = tid / 32;

    if (warp_id == 0) {
        // Load warp:
        // Load Q, K, V tiles from global memory into shared memory.
    } else if (warp_id == 1) {
        // MMA warp:
        // Issue tensor core operations for  $QK^T$  and  $PV$ .
    } else if (warp_id >= 2 && warp_id < 6) {
        // Softmax warps:
        // Compute max, exponentials, sums, and detect rescale events.
    } else if (warp_id >= 6 && warp_id < 10) {
        // Correction warps:
        // Rescale accumulated state when needed.
    } else {
        // Store warp:
        // Write output tile to global memory.
    }
}
```

This is not actual FlashAttention 4 source code. It illustrates the programming model: different warps in one CTA do different jobs.

Important hardware consequences:

- Warp specialization improves overlap between memory movement and computation.
- It allows different warp groups to focus on operations that use different hardware resources.
- It increases programming complexity because explicit synchronization is required.
- It turns the CTA into a small asynchronous pipeline.

Performance Intuition

Why FlashAttention Is Fast

FlashAttention is fast because it changes memory traffic. The naive attention pipeline writes and reads large intermediate matrices:

$$QK^T \rightarrow \text{HBM}, \quad \text{softmax}(QK^T) \rightarrow \text{HBM}.$$

FlashAttention instead keeps intermediate tiles on-chip:

$$Q_b K_t^T \rightarrow \text{online softmax update} \rightarrow \text{partial output update.}$$

The major reduction is avoiding the $N_q N_k$ intermediate write and read. The algorithm still performs approximately the same mathematical work, but it moves far fewer bytes through HBM.

Why FlashAttention 4 Needs More Pipeline Complexity

Blackwell tensor cores are extremely fast. If the kernel waits for memory loads, softmax exponentials, or correction synchronization, tensor cores will go idle. Therefore the kernel must overlap many activities:

- Loading future tiles.
- Computing current QK^T tiles.
- Performing softmax on recent score tiles.
- Correcting accumulated state if needed.
- Computing PV .
- Storing completed output tiles.

This is why FlashAttention 4 moves from a simpler producer-consumer pipeline to a more deeply specialized set of warp roles.

Softmax Becomes More Important as Matmul Gets Faster

Earlier attention kernels were often dominated by the matrix multiply pipeline. On Blackwell, tensor cores are faster and easier to drive in certain ways, so non-matmul operations become more visible.

The softmax path includes:

- Row maxima.
- Exponentiation.
- Summation.
- Scaling.
- Rescaling old output state.

Unlike matrix multiplication, this work does not run on tensor cores. Therefore, as tensor cores improve, softmax overhead becomes a larger fraction of the critical path. FlashAttention 4 responds with:

- Less frequent rescaling.
- Software exponentiation to reduce SFU bottlenecks.

- Dedicated softmax and correction warps.

Why Thresholded Rescaling Helps

Classic online softmax rescales every time the row maximum increases. But early in a row, the running maximum may change many times by small amounts. Each update can force coordination.

FlashAttention 4 observes that the purpose of subtracting the maximum is numerical stability. If the maximum changes slightly, it may be safe to delay the correction. The thresholded condition

$$m_t - m_{\text{effective}} > \tau$$

reduces correction frequency.

The performance win comes from avoiding synchronization-heavy pipeline disruption, not merely from saving a multiply.

Why Software Exponentiation Helps

If exponentials bottleneck on SFUs, then tensor cores and CUDA cores may have unused capacity. Replacing some SFU exponentials with polynomial approximations using CUDA-core FMA instructions redistributes work:

$$\text{SFU pressure} \rightarrow \text{CUDA core FMA pressure}.$$

This is useful when SFUs are the limiting resource. It may not be useful when SFUs are not saturated, which is why FlashAttention 4 appears to apply it conditionally.

Paged KV Cache and Attention Sinks

The lecture notes that FlashAttention 4 supports paged key-value tensors and attention sinks.

Paged KV cache support matters for inference systems because a long-running language model server stores past keys and values for many requests. The logical KV tensor may be laid out in pages rather than one contiguous block. The attention kernel must translate logical sequence positions into physical cache pages.

Attention sinks are a mechanism associated with long-context models. A small set of positions can act as persistent anchors or sink tokens. In practice, this lets models handle long contexts in memory-constrained settings by retaining special positions that many tokens may attend to.

These features make the kernel relevant for modern LLM inference infrastructure, even though the discussed version is BF16 forward-only.

Common Misconceptions

Misconception: FlashAttention Is Approximate Attention

FlashAttention is not approximate attention. It computes exact attention up to floating-point arithmetic differences. Its main trick is changing the order of computation and memory movement, not approximating the attention matrix.

Misconception: Softmax Is Easy Because It Is Just Pointwise Exponentials

Softmax is not purely pointwise. It contains reductions:

$$m = \max_j z_j, \quad \ell = \sum_j \exp(z_j - m).$$

In tiled attention, these reductions must be maintained online while preserving numerical stability.

Misconception: Rescaling Is Cheap Enough to Ignore

A single multiplication is cheap. A rescale event inside a warp-specialized asynchronous pipeline is not just a multiplication. It requires detection, communication, synchronization, and correction of accumulated state.

Misconception: Faster Tensor Cores Solve Everything

Faster tensor cores make the non-tensor-core parts more important. Softmax, exponentiation, reductions, memory movement, and synchronization can become bottlenecks when matrix multiplication gets faster.

Misconception: Blackwell Kernel Programming Is Just Hopper Kernel Programming with Bigger Numbers

Blackwell changes the programming model. Tensor memory, fifth-generation tensor core instructions, and different MMA behavior affect how kernels are organized. Hopper's warp-group-centric style is less central in FlashAttention 4, while deeper warp-specialized pipelines become more important.

Misconception: A Compiler Can Automatically Discover All of FlashAttention 4

Some pieces may be compiler-friendly, such as choosing between exponentiation implementations based on resource pressure. But a five-stage attention-specific pipeline with thresholded online softmax correction depends heavily on algorithmic knowledge. The speaker suggests that this is difficult for a compiler to infer from low-level thread code alone.

Practical Takeaways

- FlashAttention 4 is best understood as a Blackwell-specific evolution of tiled online attention.
- The kernel is forward-only in the discussed version and currently BF16-focused.
- The central algorithmic invariant is still exact online softmax over streamed K, V tiles.
- Blackwell tensor memory adds another important on-chip storage level for tensor-core results.
- The kernel uses more warp specialization than FlashAttention 3.
- The pipeline separates load, MMA, softmax, correction, and store responsibilities.
- Softmax is increasingly important because tensor cores are so fast.
- Rescaling is deferred until the maximum changes enough to matter numerically.
- Software exponentiation reduces SFU pressure by using CUDA-core FMA instructions.
- Modern GPU matmul-like kernels are becoming tile-centric rather than thread-centric.
- Future DSLs and compilers must expose or infer tile pipelines, synchronization, and warp specialization.

Project or Experiment Ideas

Experiment 1: Naive Attention Versus Online Attention

Implement two PyTorch versions:

1. Naive attention that materializes QK^T .
2. Tiled online attention in Python or Triton-style pseudocode.

Measure peak memory usage as sequence length increases. The expected result is that naive attention has quadratic memory growth from the materialized score matrix, while online attention avoids that intermediate.

Experiment 2: Online Softmax Numerical Stability

Create random score vectors with large positive values. Compare:

- Naive softmax without subtracting max.

- Stable softmax with subtracting max.
- Online softmax over chunks.

Verify that naive softmax can overflow, while stable and online softmax remain valid.

Experiment 3: Rescale Frequency

Simulate online softmax over random score streams and count how often the running maximum increases:

$$\Delta_t > 0.$$

Then count how often it increases by more than a threshold:

$$\Delta_t > \tau.$$

Plot correction count versus τ . This gives intuition for why FlashAttention 4 can reduce rescaling events.

Experiment 4: Polynomial Exponential Approximation

Implement a cubic approximation to 2^r on $r \in [0, 1]$. Compare error against `torch.exp2`. Then evaluate how error changes when values are cast to BF16.

Experiment 5: Warp Specialization Mental Model

Write a toy CUDA kernel where different warps in one block perform different roles:

- Warp 0 loads data into shared memory.
- Warp 1 computes a reduction.
- Warp 2 writes results.

Use barriers to coordinate. This will not be a high-performance FlashAttention kernel, but it will teach the control-flow style used by warp-specialized kernels.

Experiment 6: Nsight Compute Profiling

Profile attention-like kernels and inspect:

- Tensor core utilization.
- SFU utilization.
- Shared-memory bandwidth.
- Register pressure.
- Occupancy.

- Warp stall reasons.

Try to identify whether the kernel is bottlenecked by tensor cores, memory, SFUs, or synchronization.

Final Mental Model

FlashAttention 4 is not just “FlashAttention on a newer GPU.” It is a redesign of the attention kernel around Blackwell’s hardware balance. The algorithm still streams over K, V tiles and maintains exact online softmax state, but the implementation becomes a deeply pipelined, warp-specialized machine inside each CTA.

The best mental model is:

A FlashAttention 4 CTA is a miniature asynchronous factory. Load warps bring tiles from HBM into shared memory. MMA warps use tensor cores and tensor memory to produce score and output fragments. Softmax warps normalize scores while watching numerical stability. Correction warps rescale accumulated state only when the maximum changes enough to matter. Store warps write finished output tiles. The whole machine exists to keep Blackwell’s tensor cores busy while preventing softmax, memory movement, SFU exponentiation, and synchronization from becoming the bottleneck.

The mathematical invariant remains:

$$O = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right) V.$$

The engineering achievement is computing this expression without materializing the attention matrix, while mapping the remaining work carefully onto Blackwell’s tensor cores, CUDA cores, SFUs, tensor memory, shared memory, registers, and asynchronous warp pipeline.